

Launch Readiness

Alpha-to-Beta Gate Checklist • Progressive Web App • Cloudflare R2 CDN for 3D Assets • Zero-Downtime Deployment

Author: Daniel **Date:** Friday, 18 September 2026 **Version:** 1.0 **Series:** GPL-3.0 • Free-Tier Only • Open Source



About This Series

HavenWorld is a browser-based 3D social virtual world built on a pnpm monorepo. This is Part 9A of a 10-part build series. Parts 1–8 are complete and cover: infrastructure provisioning, the Babylon.js 3D client, authentication, Socket.io real-time networking, Phase 3 game features, automated testing, and a full security audit. Part 9A covers the transition from closed alpha (~25 invite-only testers) to open public beta, plus three critical infrastructure upgrades required at public scale.

Stack reminder: apps/client (Babylon.js 7 + Vite 6 + TypeScript → Cloudflare Pages) • apps/server (Express 5 + Socket.io 4 + Prisma + PostgreSQL 16 + Redis 7 → Oracle Cloud ARM VM via PM2 + Nginx) • packages/shared (shared types + event constants) • CI/CD: GitHub Actions.

Table of Contents

Section 1 — The Alpha → Beta Gate System

1.1 What the Gate Is

1.2 Gate Category 1 — Infrastructure Stability

1.3 Gate Category 2 — Application Quality

1.4 Gate Category 3 — Game Loop Quality

1.5 Gate Category 4 — Community & Legal Readiness

1.6 Executing the Gate (shell script)

1.7 Removing the Alpha Invite Wall

Section 2 — Progressive Web App (PWA)

2.1 Why PWA for HavenWorld • 2.2 Web App Manifest • 2.3 Creating PWA Icons • 2.4 Service Worker with Workbox • 2.5 Offline Experience • 2.6 Install Prompt • 2.7 Cloudflare Pages PWA Headers • 2.8 Verify PWA Installation

Section 3 — Cloudflare R2 CDN for 3D Assets

3.1 Why R2 • 3.2 Create the R2 Bucket • 3.3 Configure R2 API Credentials • 3.4 Install rclone • 3.5 Asset Upload Script • 3.6 Update RoomLoader • 3.7 Remove Assets from Git • 3.8 Asset Download Script • 3.9 R2 Upload in CI/CD

Section 4 — Zero-Downtime Deployment

4.1 The Problem • 4.2 PM2 Cluster Mode • 4.3 Nginx Upstream • 4.4 Socket.io Sticky Sessions • 4.5 Updated Deployment Script • 4.6 Health Check Enhancement • 4.7 Deployment Runbook

Section 5 — Part 9A Completion Checklist

Section 1 — The Alpha → Beta Gate System

1.1 What the Gate Is

The Alpha-to-Beta Gate is a formal, documented go/no-go checklist that must reach **100% pass** before the `ALPHA_INVITE_ONLY` environment flag is removed and HavenWorld is opened to the general public. Each gate item carries a specific, measurable pass condition and a concrete verification method — either an exact terminal command, a dashboard URL, or a manual QA procedure attributed to a named tester.

⚠️ Why Gates Matter: The One-Shot First Impression

Launching too early is a permanent, largely irreversible mistake. A bad launch day teaches the search index that your domain correlates with user bounces and poor engagement signals, suppressing organic discovery for months. Players who encounter a broken or laggy experience on day one rarely return — research on consumer habit formation consistently shows that the first session is the single highest-leverage moment in a game's lifecycle. A player who bounces on day one and returns on day three is a different, harder conversion than a player who had a smooth first session. The gate process exists to make day one exceptional, not just functional.

The gate is divided into four categories, each addressing a different dimension of readiness. All four must pass simultaneously before beta launch proceeds. There are no partial passes.

1.2 Gate Category 1 — Infrastructure Stability LINUX

All commands below are run on the Oracle Cloud ARM VM via SSH (`ssh ubuntu@[ORACLE_IP]`) unless otherwise noted. The pass/fail column represents the minimum threshold for launch approval.

Gate Item	Pass Condition	How to Verify
Server uptime — last 14 days	≥ 99.5% (max 1h 41m downtime)	UptimeRobot dashboard → Reports → Export 14-day CSV. Or: UptimeRobot public status page URL in LAUNCH_GATE.md.
PM2 unplanned restarts — last 7 days	Zero unplanned restarts	<code>pm2 logs havenworld-server --lines 500 grep "restart"</code> Expected output: no matches.
PostgreSQL slow query log	No queries > 1000ms	<code>sudo grep "duration:" /var/log/postgresql/postgresql-16-main.log \ awk</code>

Gate Item	Pass Condition	How to Verify
		'{print \$NF}' \ awk -F. '{if (\$1+0 > 1000) print \$0 " ms SLOW"}'Expected output: empty.
Redis memory usage	< 70% of maxmemory	redis-cli -a "\$REDIS_PASSWORD" info memory \ grep -E "used_memory_human maxmemory_human"
Oracle VM disk space	/dev/sda < 70% used	df -h /
SSL certificate validity	> 60 days remaining	sudo certbot certificates grep "Expiry Date"
GitHub Actions — last 10 commits on main	All workflows green	GitHub → Actions tab → filter by branch:main. All run statuses must show green checkmark.
k6 load test — 50 concurrent users	p95 response < 500ms, zero errors	npx k6 run load-tests/ws-concurrent.js --vus 50 --duration 60sRun from local Mac; test hits Oracle VM.
Nginx rate limiting	Returns HTTP 429 on abuse pattern	node security-tests/brute-force.jsMust observe 429 Too Many Requests after threshold.
fail2ban active with jail rules	sshd jail active and enabled	sudo fail2ban-client status sshdVerify: Status = active, Filter = sshd.

1.3 Gate Category 2 — Application Quality MAC

Run all commands from the monorepo root on your Mac development machine. Commands assume pnpm is installed and node_modules are up to date.

Gate Item	Pass Condition	How to Verify
Jest server test suite	100% pass rate, coverage ≥ 60%	pnpm --filter server test --coverageCoverage thresholds enforced in jest.config.ts.

Gate Item	Pass Condition	How to Verify
Vitest client test suite	100% pass rate, coverage $\geq$ 55%	<code>pnpm --filter client test --coverage</code>
Playwright E2E full-flow spec	Green on Chromium + Firefox	<code>pnpm --filter client test:e2e</code>
npm security audit	Zero HIGH or CRITICAL vulnerabilities	<code>npm audit --audit-level=high</code> Expected: <code>found 0 vulnerabilities</code>
TypeScript strict mode — server	Zero tsc errors	<code>pnpm --filter server tsc --noEmit</code>
TypeScript strict mode — client	Zero tsc errors	<code>pnpm --filter client tsc --noEmit</code>
Client bundle size	Total dist/ < 8MB	<code>pnpm --filter client build && du -sh apps/client/dist/</code>
Lighthouse — Performance score	$\geq$ 75	<code>npx lighthouse https://havenworld.pages.dev \ --output=json \ jq '.categories.performance.score * 100'</code>
Lighthouse — Accessibility score	$\geq$ 90	<code>npx lighthouse https://havenworld.pages.dev \ --output=json \ jq '.categories.accessibility.score * 100'</code>

1.4 Gate Category 3 — Game Loop Quality MAC + LINUX

Each item below must be verified by **at least 3 alpha testers** independently AND pass an automated Playwright assertion where noted. Manual testers should record their name and date in `LAUNCH_GATE.md`.

Feature Under Test	Pass Condition	Verification Method
Registration → email verify → login	< 60 seconds end-to-end	Playwright timer assertion + manual tester stopwatch. Test on a fresh incognito window.

Feature Under Test	Pass Condition	Verification Method
Room join time	< 5 seconds click → fully loaded on WiFi	Browser Network tab → measure time from navigation to last GLB response complete. Playwright: <code>waitForSelector('.room-loaded')</code> with 5s timeout.
Avatar customization persistence	Saves and reloads correctly after hard refresh (Ctrl+Shift+R)	Manual: customize avatar, hard refresh, verify same appearance. Playwright: <code>set appearance</code> → reload page → assert DOM values match.
Chat message round-trip latency	< 250ms local, < 600ms cross-country	Socket.io client timestamps on send and receive. Log delta in browser console. Test with a tester in a different US region.
Player movement sync	No visible rubber-banding at 25 concurrent users in same room	Manual: 3 testers in same room, all moving simultaneously. Record 60-second screen capture. Review for rubber-band artifacts.
P2P trade flow	10 consecutive trades complete without error between 2 test accounts	Manual: run 10 trades using account A and account B. Verify item counts in DB after each trade: <code>SELECT * FROM inventory WHERE userId IN (A, B)</code> .
Fishing mini-game — coin integrity	No coin duplication in 50 consecutive catches	# [LINUX] — run after 50 catches <code>psql \$DATABASE_URL -c \"SELECT SUM(coins) FROM users WHERE id IN (test_ids);\"</code> Compare before/after sum. Delta must equal exactly expected catch reward * 50.
Workshop crafting — timer accuracy	Items delivered within ±5 seconds of scheduled completion	Manual: start 5 craft jobs with known durations. Record actual delivery timestamps. All must be within 5s of expected.
Pet AI — memory stability	No infinite loop or memory leak after 30 minutes in room	Chrome DevTools → Memory tab → record heap snapshots at 0, 15, 30 minutes. Heap must not grow unboundedly (< 20% increase acceptable).

1.5 Gate Category 4 — Community & Legal

Readiness MAC

Item	Pass Condition	How to Verify
Terms of Service	Published and accessible	Visit https://havenworld.pages.dev/terms — must load with complete ToS text, no 404.
Privacy Policy	Published and accessible	Visit https://havenworld.pages.dev/privacy — must load with complete Privacy Policy, no 404.
Discord server	3 active channels, pinned rules post	Verify #announcements, #bug-reports, #general exist. Pinned message in #general with community rules is visible to new members.
Hashnode dev blog	Minimum 3 published posts	Visit Hashnode profile → count published articles. All must be public, not drafts.
README.md	Complete: setup, tech stack, license, Discord link	Open cat README.md from repo root. Check for presence of: quick start, tech stack table, GPL-3.0 badge, Discord invite link.
ROADMAP.md	Accurate and up to date	Daniel manually reviews ROADMAP.md — all completed items marked [DONE], Part 9 items present.
SECURITY.md	Vulnerability disclosure email working	Send a test email to the address in SECURITY.md — confirm receipt in inbox within 5 minutes.
Admin account	Daniel's account is role=ADMIN in production DB	# [LINUX] on Oracle VM psql \$DATABASE_URL -c \ "SELECT id, email, role FROM users \ WHERE email = 'your@email.com';"Expect ed: role = ADMIN.

Item	Pass Condition	How to Verify
Discord Moderator	At least 1 trusted @Moderator with in-game mod role	Discord → Server Settings → Members. Verify @Moderator role assigned. In-game: SELECT role FROM users WHERE discordId = 'mod_id' returns MODERATOR.

1.6 Executing the Gate MAC + LINUX

Create the following shell script at `scripts/beta-gate-check.sh` in the repository root. This script automates all programmatically-verifiable gate items. Manual items are listed separately at the end of the run.

```
#!/bin/bash # scripts/beta-gate-check.sh # HavenWorld Alpha-to-Beta Automated Gate Check # Run from repo root on Mac. Requires ORACLE_HOST env var set. # Usage: bash scripts/beta-gate-check.sh set -e echo
"===== " echo "|| HavenWorld Beta Gate Check - $(date +%Y-%m-%d) ||" echo
"===== " echo "" PASS=0; FAIL=0
check() { local name="$1" local cmd="$2" local expect="$3" local result result=$(eval "$cmd" 2>&1) || true if echo "$result" | grep -q "$expect"; then echo "✅ $name" ((PASS++)) else echo "❌ $name" echo "Expected pattern : $expect" echo "Got : $(echo "$result" | tail -2)" ((FAIL++)) fi } echo "— Category 2: Application Quality =====" # TypeScript: server (no errors means tsc exits 0 with no output) TS_SERVER=$(cd apps/server && npx tsc --noEmit 2>&1 | wc -l | tr -d ' ') if [ "$TS_SERVER" = "0" ]; then echo "✅ Server TypeScript - zero errors"; ((PASS++)) else echo "❌ Server TypeScript - $TS_SERVER error line(s)"; ((FAIL++)) fi # TypeScript: client TS_CLIENT=$(cd apps/client && npx tsc --noEmit 2>&1 | wc -l | tr -d ' ') if [ "$TS_CLIENT" = "0" ]; then echo "✅ Client TypeScript - zero errors"; ((PASS++)) else echo "❌ Client TypeScript - $TS_CLIENT error line(s)"; ((FAIL++)) fi # Jest server tests check "Server Jest suite" \ "pnpm --filter server test --passWithNoTests 2>&1 | tail -4" \ "passed" # Vitest client tests check "Client Vitest suite" \ "pnpm --filter client test --passWithNoTests 2>&1 | tail -4" \ "passed" # npm audit check "npm audit (HIGH/CRITICAL = 0)" \ "npm audit --audit-level=high 2>&1 | tail -3" \ "found 0" # Bundle size: must be under 8MB echo "" echo "— Bundle Size Check =====" pnpm --filter client build --silent 2>&1 | tail -1 BUNDLE_KB=$(du -sk apps/client/dist/ | cut -f1) BUNDLE_MB=$((BUNDLE_KB / 1024)) if [ "$BUNDLE_MB" -lt 8 ]; then echo "✅ Client bundle size - ${BUNDLE_MB}MB (< 8MB)"; ((PASS++)) else echo "❌ Client bundle size - ${BUNDLE_MB}MB (exceeds 8MB limit)"; ((FAIL++)) fi echo "" echo "— Remote Infrastructure Checks (SSH) =====" if [ -z "$ORACLE_HOST" ]; then echo "⚠️ ORACLE_HOST not set - skipping SSH checks" else # SSL cert days remaining SSL_DAYS=$(ssh ubuntu@"$ORACLE_HOST" \ "sudo certbot certificates 2>/dev/null | grep 'Expiry Date'" \ | grep -oP '\d{4}-\d{2}-\d{2}' \ | xargs -I{} bash -c \ 'echo $(( ( $(date -d "{}" +%s) - $(date +%s) ) / 86400 ))' 2>/dev/null || echo 0) if [ "${SSL_DAYS:-0}" -gt 60 ]; then echo "✅ SSL certificate - $SSL_DAYS days remaining"; ((PASS++)) else echo "❌
```

```

SSL certificate - $SSL_DAYS days remaining (need > 60)"; ((FAIL++))    fi    #
PM2 unplanned restarts    RESTARTS=$(ssh ubuntu@"$ORACLE_HOST" \    "pm2 logs
havenworld-server --lines 500 --nostream 2>&1 | grep -c 'restart'" || echo 0)
if [ "${RESTARTS:-0}" -eq 0 ]; then    echo "  PM2 restarts in last 500
log lines - none"; ((PASS++))    else    echo "  PM2 restarts detected -
$RESTARTS occurrence(s)"; ((FAIL++))    fi    # Disk space    DISK_PCT=$(ssh
ubuntu@"$ORACLE_HOST" \    "df / | tail -1 | awk '{print \$5}' | tr -d '%'"
|| echo 100)    if [ "${DISK_PCT:-100}" -lt 70 ]; then    echo "  Disk
space - ${DISK_PCT}% used (< 70%)" ; ((PASS++))    else    echo "  Disk
space - ${DISK_PCT}% used (exceeds 70%)" ; ((FAIL++))    fi fi echo "" echo
"— Manual Checks Required (record in LAUNCH_GATE.md) " echo " [ ]
UptimeRobot 14-day report: ≥ 99.5% uptime" echo " [ ] k6 50-VU load test:
p95 < 500ms, zero errors" echo " [ ] Playwright E2E: green on Chromium +
Firefox" echo " [ ] Lighthouse: Performance ≥ 75, Accessibility ≥ 90" echo "
[ ] Game loop QA: completed by 3 named testers" echo " [ ] Legal pages:
/terms and /privacy live" echo " [ ] Discord: 3 channels, pinned rules, 1
moderator" echo " [ ] Daniel's account: role=ADMIN in production DB" echo ""
echo "===== " echo "
Automated: $PASS passed, $FAIL failed" echo
"===== " if [ "$FAIL" -eq
0 ]; then    echo "  All automated checks passed."    echo " Complete
manual checks above, then proceed to beta." else    echo "  $FAIL
automated check(s) failed. Fix before beta."    exit 1 fi

```

LAUNCH_GATE.md — The Living Gate Document

Create LAUNCH_GATE.md at the repository root. This document is the official sign-off record for beta launch. Each gate item gets a checkbox, a date, and the name of the person who verified it. Example format:

```

### HavenWorld Beta Launch Gate Last updated: 2026-09-18 by Daniel ###
Category 1 – Infrastructure Stability - [x] Server uptime ≥ 99.5% – verified
2026-09-15 by Daniel (UptimeRobot: 99.87%) - [x] PM2 no unplanned restarts –
verified 2026-09-15 by Daniel - [ ] PostgreSQL no slow queries – PENDING -
[ ] Redis memory < 70% – PENDING ... ### Category 2 – Application Quality -
[x] Server Jest 100% pass – verified 2026-09-18 by Daniel (CI run #147) ...
### GO / NO-GO DECISION Date: _____ Decision: _____
Authorized by: Daniel

```

1.7 Removing the Alpha Invite Wall MAC + LINUX

Once every gate item is checked and signed off in LAUNCH_GATE.md, perform the following steps in order. Do not skip or reorder them.

```

ssh ubuntu@$ORACLE_HOST sudo nano /opt/havenworld/apps/server/.env # Change:
ALPHA_INVITE_ONLY=true # To:    ALPHA_INVITE_ONLY=false # Save and exit
(Ctrl+X, Y, Enter)
pm2 reload ecosystem.config.js --update-env # Verify the env var was picked
up: pm2 env 0 | grep ALPHA

```

```
// Before async register(username: string, email: string, password: string,
inviteCode: string) // After async register(username: string, email: string,
password: string)
git add -A git commit -m "feat: open public beta – remove alpha invite wall"
git push origin main
```

🌐 HavenWorld is now open to the public! After months of closed alpha testing with our amazing early community, HavenWorld is officially in open public beta. No invite code needed – just visit <https://havenworld.pages.dev> and create your account. Make friends. Decorate your loft. Explore Haven. 🏠

✨ – Daniel

Section 2 — Progressive Web App (PWA) MAC

2.1 Why PWA for HavenWorld

A Progressive Web App allows players to "install" HavenWorld directly from their browser to their desktop taskbar, macOS Dock, Android home screen, or iOS home screen — with no app store submission, no review process, and no fees. From the player's perspective it behaves exactly like a native app. From the developer's perspective it is just a set of web standards built into the browser.

PWA Benefit	HavenWorld Impact
Standalone window (no browser chrome/tabs)	Maximizes usable game viewport; no accidental tab switching mid-session
App icon on home screen / taskbar	Significantly increases return visit rate vs. bookmarks
Service worker asset caching	Babylon.js engine chunk + GLB assets cached locally → faster repeat loads
Offline splash screen	Players see a polite "you're offline" page instead of a blank browser error
No app store fees or review delays	Immediate updates — push to main, players get updated code within minutes
Cross-platform (desktop + mobile)	Same codebase serves Windows, macOS, Android, iOS via PWA install

💡 Scope Note: PWA ≠ Offline Gameplay

HavenWorld is a live social world requiring a persistent WebSocket connection. PWA caching is used to speed up *loading* (static assets cached by the service worker) and to display a graceful offline page, not to enable offline gameplay. Do not configure the service worker to cache API responses or attempt to queue Socket.io events offline.

2.2 Web App Manifest MAC

Create the file `apps/client/public/manifest.webmanifest` with the following content. Cloudflare Pages will serve this file automatically from the root domain.

```
{  "name": "HavenWorld",  "short_name": "HavenWorld",  "description": "A browser-based 3D social virtual world. Make friends, decorate your loft, and explore Haven.",  "start_url": "/",  "display": "standalone",  "background_color": "#0f1b2d",  "theme_color": "#00d4c8",  "orientation": "landscape-primary",  "icons": [    {      "src": "/icons/icon-192.png",      "sizes": "192x192",      "type": "image/png",      "purpose": "any maskable"    },    {      "src": "/icons/icon-512.png",      "sizes": "512x512",      "type": "image/png",      "purpose": "any maskable"    }  ],  "screenshots": [    {      "src": "/screenshots/lobby.png",      "sizes": "1280x720",      "type": "image/png",      "form_factor": "wide"    },    {      "src": "/screenshots/room.png",      "sizes": "1280x720",      "type": "image/png",      "form_factor": "wide"    }  ],  "categories": ["games", "social"],  "lang": "en-US" }
```

Add the following tags to the `<head>` section of `apps/client/index.html`. Place them after the `<meta charset>` tag and before any script tags:

```
<!-- PWA Manifest and meta --> <link rel="manifest" href="/manifest.webmanifest" /> <meta name="theme-color" content="#00d4c8" /> <!-- iOS PWA support (Safari does not read the manifest for these) --> <meta name="apple-mobile-web-app-capable" content="yes" /> <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" /> <meta name="apple-mobile-web-app-title" content="HavenWorld" /> <link rel="apple-touch-icon" href="/icons/icon-192.png" />
```

2.3 Creating PWA Icons MAC

PWA icons must be exact pixel dimensions and PNG format. The maskable version must have the icon centered within a circle with at least 20% safe-zone padding on all sides (so that circular masking on Android does not clip the logo).

```
sips -z 192 192 apps/client/public/icons/icon-512.png \ --out
apps/client/public/icons/icon-192.png
sips -g pixelWidth -g pixelHeight apps/client/public/icons/icon-192.png #
Expected output: # pixelWidth: 192 # pixelHeight: 192
```

💡 Maskable Icon Safe Zone Visualizer

Use the free tool at **maskable.app** to preview how your icon looks when clipped to different shapes (circle, rounded square, squircle). Upload your icon-512.png and verify the logo mark is fully visible in the "minimum safe area" preview before finalizing.

2.4 Service Worker with Workbox MAC

Install the Vite PWA plugin (wraps Google Workbox, entirely free and open-source):

```
pnpm add -D vite-plugin-pwa --filter client
```

Update `apps/client/vite.config.ts` to register the PWA plugin. The configuration below uses *autoUpdate* mode (service worker updates silently in the background and activates on next page load) and implements CacheFirst strategies for large binary assets:

```
// apps/client/vite.config.ts import { defineConfig } from 'vite'; import
{ VitePWA } from 'vite-plugin-pwa'; export default defineConfig({  plugins:
[    VitePWA({      registerType: 'autoUpdate',          // Assets to
precache during install (small files only)      includeAssets:
['icons/*.png', 'screenshots/*.png', 'offline.html'],          // We manage
manifest.webmanifest ourselves – do not auto-generate      manifest: false,
workbox: {          // --- Runtime caching strategies ---
runtimeCaching: [          // GLB 3D models: Cache-first, 30-day expiry, max
50 files          // These are served from R2 (Section 3) – large, rarely
change          {            urlPattern: /\.glb$/,                handler:
'CacheFirst',            options: {              cacheName: 'glb-assets',
expiration: {                maxEntries: 50,                maxAgeSeconds:
60 * 60 * 24 * 30, // 30 days            },          },          },
// WASM binaries: Cache-first, 90-day expiry (Havok physics, etc.)
{            urlPattern: /\.wasm$/,                handler: 'CacheFirst',
options: {              cacheName: 'wasm-assets',                expiration:
{                maxEntries: 5,                maxAgeSeconds: 60 * 60 * 24
* 90, // 90 days            },          },          },          //
PNG/JPEG images: StaleWhileRevalidate (show cached, update in bg)      {
urlPattern: /\. (png|jpg|jpeg|webp) $/,                handler:
'StaleWhileRevalidate',            options: {              cacheName:
'image-assets',            expiration: { maxEntries: 100, maxAgeSeconds:
60 * 60 * 24 * 7 },          },          },          },          //
Offline fallback: navigate to /index.html for all navigation requests
```

```

navigateFallback: '/index.html', // NEVER intercept API calls or
Socket.io connection upgrade requests navigateFallbackDenylist: [
/^\/api\/$/, // Offline page
/^\/socket\.io\/$/, // Offline page
], // Offline page
shown when navigateFallback itself fails offlineGoogleAnalytics:
false, }, }, ], });

```

⚠ Do Not Cache API Calls

Never add `/api/` routes to Workbox runtime caching. Serving stale API responses (auth tokens, inventory state, room lists) will cause data corruption and security issues. The `navigateFallbackDenylist` above ensures Socket.io's HTTP long-polling fallback is never intercepted by the service worker.

2.5 Offline Experience MAC

Create the offline fallback page at `apps/client/public/offline.html`. This page is shown by the service worker when the player is offline and attempts to load the game. Note: this page intentionally uses inline styles because it is served outside the Vite build pipeline.

```

<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8" /> <meta
name="viewport" content="width=device-width, initial-scale=1.0" />
<title>HavenWorld – Offline</title> <style> * { box-sizing: border-box;
margin: 0; padding: 0; } body { background: #0f1b2d; color:
#ffffff; font-family: system-ui, -apple-system, sans-serif; min-
height: 100vh; display: flex; align-items: center; justify-
content: center; flex-direction: column; gap: 18px;
padding: 24px; text-align: center; } .logo { font-size:
3rem; } h1 { font-size: 1.8rem; color: #00d4c8; letter-spacing: -
0.5px; } p { color: #8899aa; max-width: 400px; line-height: 1.6; }
button { background: #00d4c8; color: #0f1b2d; border: none;
padding: 12px 32px; font-size: 1rem; cursor: pointer; font-
weight: bold; margin-top: 8px; } button:hover { background:
#00b8ae; } .status { font-size: 0.8rem; color: #556677; margin-top:
8px; } </style> </head> <body> <div class="logo"><img alt="HavenWorld logo" data-bbox="635 738 655 753"/></div>
<h1>HavenWorld</h1> <p> You're offline. HavenWorld is a live social
world that requires an internet connection to play. Check your connection
and try again. </p> <button onclick="window.location.reload()">Try
Again</button> <p class="status">If your connection is restored, click Try
Again to rejoin Haven.</p> </body> </html>

```

2.6 Install Prompt (PWA "Add to Home Screen")

MAC

Create the install prompt class at `apps/client/src/ui/InstallPrompt.ts`:

```
// apps/client/src/ui/InstallPrompt.ts // Manages the "Install HavenWorld"
PWA install banner. // Shows after 60 seconds of first login, never
before. // Respects user dismissal (stored in localStorage). interface
BeforeInstallPromptEvent extends Event {   prompt(): Promise<void>;
userChoice: Promise<{ outcome: 'accepted' | 'dismissed' }>; } export class
InstallPrompt {   private deferredPrompt: BeforeInstallPromptEvent | null =
null;   private promptShown = false;   constructor()
{     window.addEventListener('beforeinstallprompt', (e) => {       //
Prevent the default mini-infobar (Chrome default) from appearing
e.preventDefault();       this.deferredPrompt = e as
BeforeInstallPromptEvent;       const alreadyDeclined =
localStorage.getItem('pwa_install_declined');       if (!this.promptShown
&& !alreadyDeclined) {         // Wait 60 seconds after page load before
showing banner         // (don't interrupt the player during their first
moments in-world)         setTimeout(() => this.showBanner(),
60_000);       }     });     // Wire up the install and dismiss buttons
document.getElementById('pwa-install-btn')       ?.addEventListener('click',
() => this.install());     document.getElementById('pwa-dismiss-
btn')       ?.addEventListener('click', () => this.dismiss());   }   private
showBanner(): void {     const banner = document.getElementById('pwa-install-
banner');     if (banner) {       banner.style.display = 'flex';
this.promptShown = true;     }   }   async install(): Promise<void> {     if
(!this.deferredPrompt) return;     this.deferredPrompt.prompt();     const
{ outcome } = await this.deferredPrompt.userChoice;     if (outcome ===
'dismissed') {       // Remember the decision; don't ask again this session
localStorage.setItem('pwa_install_declined', '1');     }     this.deferredPrompt = null;     this.hideBanner();   }   dismiss(): void {
localStorage.setItem('pwa_install_declined', '1');     this.hideBanner();   }
private hideBanner(): void {     const banner = document.getElementById('pwa-
install-banner');     if (banner) banner.style.display = 'none';   } }
```

Add the install banner HTML to `apps/client/index.html`, inside `<body>` before the closing `</body>` tag:

```
<!-- PWA Install Banner – hidden by default, shown by InstallPrompt.ts -->
<div id="pwa-install-banner" style="display:none;
position:fixed;         bottom:80px;         left:50%;
transform:translateX(-50%);         background:#1a2e45;         border:1px
solid #00d4c8;         padding:14px 22px;         z-index:200;
align-items:center;         gap:14px;         white-space:nowrap;
font-family:system-ui,sans-serif;" >   <span style="color:#ffffff;font-
size:0.95rem;">   📱 Install HavenWorld for quicker access!   </span>
<button id="pwa-install-btn"
style="background:#00d4c8;color:#0f1b2d;border:none;         padding:8px
20px;cursor:pointer;font-weight:bold;font-size:0.9rem;"   >Install</button>
<button id="pwa-dismiss-btn"
style="background:none;border:none;color:#8899aa;cursor:pointer;font-
size:0.9rem;"   >Not now</button> </div>
```

Initialize `InstallPrompt` in your main entry point (`apps/client/src/main.ts`) after the user has logged in:

```
// In apps/client/src/main.ts, after login success: import { InstallPrompt }
from './ui/InstallPrompt'; // Instantiate once globally (constructor wires
up event listeners) const _installPrompt = new InstallPrompt();
```

2.7 Cloudflare Pages PWA Headers MAC

Create `apps/client/public/_headers`. Cloudflare Pages reads this file and applies the specified HTTP response headers to matching routes. This is critical for WASM cross-origin isolation and correct PWA MIME types.

```
/manifest.webmanifest Content-Type: application/manifest+json Cache-
Control: max-age=86400 /HavokPhysics.js.wasm Content-Type:
application/wasm Cross-Origin-Opener-Policy: same-origin Cross-Origin-
Embedder-Policy: require-corp /*.wasm Content-Type: application/wasm
Cross-Origin-Opener-Policy: same-origin Cross-Origin-Embedder-Policy:
require-corp /*.glb Cache-Control: max-age=2592000, public /icons/*.png
Cache-Control: max-age=604800, public / X-Frame-Options: DENY X-Content-
Type-Options: nosniff Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=() Cache-Control:
no-store /offline.html Cache-Control: max-age=86400
```

2.8 Verify PWA Installation MAC

After deploying to Cloudflare Pages, verify the PWA is correctly configured:

1. Open **Chrome** → navigate to `https://havenworld.pages.dev`.
2. Open DevTools (F12) → **Application** tab → **Manifest** panel.
3. Verify all fields populate correctly: Name = HavenWorld, Theme Color = #00d4c8, both icons show (192 and 512), no manifest errors in the Issues panel.
4. In the same Application tab → **Service Workers**: verify the SW is registered and status shows "activated and running".
5. Run Lighthouse: DevTools → Lighthouse tab → check "Progressive Web App" → Analyze. Target: **PWA score 100** (all checks green, installable, works offline).

6. Test install: in the Chrome address bar, click the install icon (⊕) → confirm
HavenWorld installs as a standalone app with its own window and taskbar icon.

Section 3 — Cloudflare R2 CDN for 3D Assets MAC + LINUX

3.1 Why R2 for 3D Assets

HavenWorld's 3D GLB files are large binary assets: individual room GLBs range from 2–15 MB, the base avatar GLB is approximately 3 MB with skeletal animations, and furniture items add up over a play session. Serving all of these from the Oracle Cloud ARM VM imposes two problems:

Problem	Impact at Alpha (25 users)	Impact at Beta (200+ users)
Oracle free-tier egress cap (~50 Mbps shared)	Manageable	Bandwidth saturated → all users experience slow room loads simultaneously
Single geographic origin (US region Oracle DC)	Negligible (all testers are local)	Players in EU/Asia experience 300–800ms additional latency per asset
GLBs in Git repo inflating history	Clone time ~seconds	Clone time minutes → contributor experience degrades

Cloudflare R2 resolves all three problems with zero egress fees, global CDN edge delivery, and S3-compatible APIs that integrate with existing tooling. The free tier allows 10 GB storage and 10 million Class B read operations per month.

3.2 Create the R2 Bucket MAC

7. Log in to **Cloudflare Dashboard** (dash.cloudflare.com) with the same account used for Cloudflare Pages.
8. Left sidebar → **R2 Object Storage** → **Create Bucket**.
9. Bucket name: `havenworld-assets`. Location: leave as *Automatic* (Cloudflare distributes globally). Click **Create Bucket**.
10. Open the `havenworld-assets` bucket → **Settings** tab → **Public Access** section.
11. Click **Allow Access** to enable public bucket access.
12. Under **Custom Domains**, click **Connect Domain** → enter:
`assets.havenworld.pages.dev`. Cloudflare automatically creates the CNAME DNS record in your zone.
13. After a moment, your asset URLs are live at the pattern:
`https://assets.havenworld.pages.dev/rooms/town_square.glb`

Cloudflare Zone Requirement

The custom domain `assets.havenworld.pages.dev` only works if your domain is managed through Cloudflare DNS (it is, since you're using Cloudflare Pages). If you want to use a custom root domain (e.g., `assets.havenworld.gg`), the domain must be added to the same Cloudflare account and have its DNS managed by Cloudflare.

3.3 Configure R2 API Credentials MAC

14. In Cloudflare Dashboard → **R2** → **Manage R2 API Tokens**.
15. Click **Create API Token**. Set permissions: *Object Read and Write*, scope to bucket `havenworld-assets` only.
16. Copy the three values shown after creation (these are shown only once):
 - **Account ID** (from R2 overview page)

- **Access Key ID**
- **Secret Access Key**

17. Add them to GitHub repository secrets: Settings → Secrets and variables → Actions → New repository secret:

- R2_ACCOUNT_ID
- R2_ACCESS_KEY_ID
- R2_SECRET_ACCESS_KEY

3.4 Install rclone for Asset Upload MAC

rclone is a free, open-source command-line program that syncs files to/from cloud storage. It supports S3-compatible APIs (Cloudflare R2 is S3-compatible) and only uploads files that have changed since the last sync.

```
# [MAC] Install rclone via Homebrew brew install rclone # Verify installation rclone version
```

Configure rclone with the R2 remote interactively:

```
# [MAC] Interactive configuration rclone config
```

When prompted, enter the following responses:

Prompt	Your Response
n) New remote or...	n
name>	havenworld-r2
Storage> (type number for s3)	5 (Amazon S3 Compliant Storage Providers)
provider>	Cloudflare (R2 Storage)
Access Key ID	[paste R2_ACCESS_KEY_ID]
Secret Access Key	[paste R2_SECRET_ACCESS_KEY]
Endpoint	https://[R2_ACCOUNT_ID].r2.cloudflarestorage.com
All other prompts	Press Enter (accept defaults)
y) Yes this is OK	y
q) Quit config	q

Test the connection:

```
# Should return empty output (no files yet) with no errors rclone ls
havenworld-r2:havenworld-assets echo "Exit code: $?" # Should be 0
```

3.5 Asset Upload Script MAC

Create `scripts/upload-assets.sh` at the repository root:

```
#!/bin/bash # scripts/upload-assets.sh # Upload all 3D assets to Cloudflare
R2. # Syncs only changed files (rclone compares checksums). # Run: bash
scripts/upload-assets.sh # Requires: rclone installed and configured with
havenworld-r2 remote. set -euo pipefail BUCKET="havenworld-r2:havenworld-
assets" LOCAL="apps/client/public/assets" if ! command -v rclone
&>/dev/null; then echo "❌ rclone not found. Run: brew install rclone"
exit 1 fi echo "🔄 Uploading HavenWorld assets to Cloudflare R2..." echo "
Source : $LOCAL" echo "   Bucket : $BUCKET" echo "" # Rooms – exclude source
.blend files echo "   → Rooms..." rclone sync "$LOCAL/rooms" "$BUCKET/rooms" \
--exclude "*.blend" \ --exclude "*.blend1" \ --progress \ --transfers 4
\ --checksum # Avatars echo "   → Avatars..." rclone sync "$LOCAL/avatars"
"$BUCKET/avatars" \ --progress --transfers 4 --checksum # Furniture echo "
→ Furniture..." rclone sync "$LOCAL/furniture" "$BUCKET/furniture" \ --
progress --transfers 4 --checksum # Tilesets (PNG spritesheets for 2D UI
elements) echo "   → Tilesets..." rclone sync "$LOCAL/tilesets"
"$BUCKET/tilesets" \ --progress --transfers 4 --checksum echo "" echo "✅
Upload complete!" echo "   Rooms      :
https://assets.havenworld.pages.dev/rooms/" echo "   Avatars      :
https://assets.havenworld.pages.dev/avatars/" echo "   Furniture   :
https://assets.havenworld.pages.dev/furniture/"
```

Add to package.json root scripts:

```
{  "scripts": {    "upload:assets": "bash scripts/upload-assets.sh",
  "download:assets": "bash scripts/download-assets.sh"  } }
```

3.6 Update RoomLoader to Use R2 URL MAC

Update `apps/client/src/world/RoomLoader.ts` to read the asset base URL from a Vite environment variable. In development, assets are served from the local `/assets` folder. In production, they are served from Cloudflare R2.

```
// apps/client/src/world/RoomLoader.ts import * as BABYLON from
'@babylonjs/core'; // In development: falls back to /assets (local
folder) // In production: reads VITE_ASSET_BASE_URL from .env.production //
= https://assets.havenworld.pages.dev const ASSET_BASE =
import.meta.env.VITE_ASSET_BASE_URL ?? '/assets'; /** * Load a room's GLB
mesh into the scene. * @param scene - Active Babylon.js scene * @param
roomId - Room identifier (e.g., "town_square", "user_loft_42") */ export
async function loadRoom(  scene: BABYLON.Scene,  roomId: string ):
Promise<BABYLON.AbstractMesh[]> {  const glbUrl = `${ASSET_BASE}/rooms/$
{roomId}.glb`;  console.log(`[RoomLoader] Loading: ${glbUrl}`);  const
```

```
result = await BABYLON.SceneLoader.ImportMeshAsync('', glbUrl, '', scene);
console.log(`[RoomLoader] Loaded ${result.meshes.length} meshes for room: ${
roomId}`); return result.meshes; } /** Returns the URL of the base avatar
GLB. */ export function getAvatarUrl(): string { return
`${ASSET_BASE}/avatars/base_avatar.glb`; } /** Returns the URL of a specific
furniture item GLB. */ export function getFurnitureUrl(itemId: string):
string { return `${ASSET_BASE}/furniture/${itemId}.glb`; }
```

Update the environment files:

```
# apps/client/.env.development VITE_SERVER_URL=http://localhost:3000
VITE_ASSET_BASE_URL=/assets
# apps/client/.env.production VITE_SERVER_URL=https://[oracle-ip-with-
dashes].nip.io VITE_ASSET_BASE_URL=https://assets.havenworld.pages.dev
```

Add the Cloudflare Pages environment variable via Dashboard: Cloudflare → Pages → havenworld → Settings → Environment Variables → Add variable:

- Name: `VITE_ASSET_BASE_URL`
- Value: `https://assets.havenworld.pages.dev`
- Environment: Production

3.7 Remove Assets from Git MAC

Binary GLB files do not belong in a Git repository. They are opaque binary blobs that cannot be diffed, inflate clone size dramatically, and persist in history forever even after deletion. Remove them now and move them permanently to R2 as the source of truth.

First, append to `.gitignore` at the repository root:

```
# 3D Assets – stored in Cloudflare R2, not Git. # Download locally with: bash
scripts/download-assets.sh apps/client/public/assets/rooms/*.glb
apps/client/public/assets/avatars/*.glb
apps/client/public/assets/furniture/*.glb
apps/client/public/assets/tilesets/*.png
```

If any GLB files were already committed to the repository history, remove them completely using `git filter-repo` (this rewrites history — coordinate with all contributors before doing this):

```
# [MAC] – Install git-filter-repo (one-time setup) pip3 install git-filter-
repo # Remove all .glb files from every commit in history # Run from repo
root – WARNING: this rewrites history git filter-repo \ --path-glob
'apps/client/public/assets/**/*.glb' \ --invert-paths # Force-push the
rewritten history to all branches git push origin --force --all git push
origin --force --tags
```

⚠️ Coordinate History Rewrite With All Contributors

Running `git filter-repo` rewrites commit SHAs for every commit after the first GLB was added. All contributors must delete their local clones and re-clone after this operation. Announce in Discord before executing. After re-cloning, all contributors must run `bash scripts/download-assets.sh` to get the assets locally.

3.8 Asset Download Script for New Developers MAC

Create `scripts/download-assets.sh` so new contributors can get 3D assets without needing them in Git:

```
#!/bin/bash # scripts/download-assets.sh # Download all 3D assets from
Cloudflare R2 for local development. # Run after first clone: bash
scripts/download-assets.sh # Requires: rclone installed and configured with
havenworld-r2 remote. set -euo pipefail BUCKET="havenworld-r2:havenworld-
assets" LOCAL="apps/client/public/assets" if ! command -v rclone
&>/dev/null; then echo "❌ rclone not found." echo "    Install: brew
install rclone" echo "    Configure: rclone config (see
docs/CONTRIBUTING.md)" exit 1 fi echo "📁 Downloading HavenWorld assets
from Cloudflare R2..." mkdir -p "$LOCAL/{rooms,avatars,furniture,tilesets}"
rclone sync "$BUCKET/rooms" "$LOCAL/rooms" --progress --transfers 4
rclone sync "$BUCKET/avatars" "$LOCAL/avatars" --progress --transfers 4
rclone sync "$BUCKET/furniture" "$LOCAL/furniture" --progress --transfers 4
rclone sync "$BUCKET/tilesets" "$LOCAL/tilesets" --progress --transfers 4
echo "" echo "✅ Assets ready for local development at: $LOCAL"
```

Document this in `README.md` under the Getting Started section:

```
## Getting Started 1. Clone the repository: `git clone
https://github.com/you/havenworld` 2. Install dependencies: `pnpm install`
3. Download 3D assets: `bash scripts/download-assets.sh` (Requires
rclone — see docs/CONTRIBUTING.md for setup) 4. Copy environment files: `cp
apps/client/.env.example apps/client/.env.development` 5. Start development:
`pnpm dev`
```

3.9 R2 Upload in CI/CD (Automated Asset Sync) MAC

Add the following job step to `.github/workflows/deploy-client.yml`. This step runs on every push to `main` and uploads any new or changed assets to R2. The `rclone --checksum` flag ensures unchanged files are never re-uploaded:

```
# Add this step BEFORE the Cloudflare Pages deploy step - name: Upload
3D Assets to Cloudflare R2 env:
RCLONE_CONFIG_HAVENWORLD_R2_TYPE: s3
RCLONE_CONFIG_HAVENWORLD_R2_PROVIDER: Cloudflare
RCLONE_CONFIG_HAVENWORLD_R2_ACCESS_KEY_ID: ${ secrets.R2_ACCESS_KEY_ID }
RCLONE_CONFIG_HAVENWORLD_R2_SECRET_ACCESS_KEY: $
{{ secrets.R2_SECRET_ACCESS_KEY }}
RCLONE_CONFIG_HAVENWORLD_R2_ENDPOINT: https://$
{{ secrets.R2_ACCOUNT_ID }}.r2.cloudflarestorage.com
RCLONE_CONFIG_HAVENWORLD_R2_NO_CHECK_BUCKET: "true" run: |
# Install rclone on the GitHub Actions Ubuntu runner sudo apt-get
install -y rclone # Sync all asset subdirectories (only uploads
changed files) rclone sync apps/client/public/assets/rooms \
havenworld-r2:havenworld-assets/rooms \ --exclude "*.blend" --
exclude "*.blend1" \ --checksum --transfers 4 rclone
sync apps/client/public/assets/avatars \ havenworld-
r2:havenworld-assets/avatars \ --checksum --transfers 4
rclone sync apps/client/public/assets/furniture \ havenworld-
r2:havenworld-assets/furniture \ --checksum --transfers 4
echo "✅ R2 asset sync complete"
```

Section 4 — Zero-Downtime Deployment LINUX

4.1 The Problem with the Current Deployment

The deployment pipeline from Part 4C performs the following steps in sequence:

```
git pull pnpm install npx prisma migrate deploy pnpm build pm2 restart
havenworld-server # <-- The problem
```

pm2 restart performs a hard stop-and-start of the Node.js process. During the brief window between stop and start (typically 1–3 seconds on the Oracle ARM VM), the following failures occur:

Failure Mode	Impact at Alpha (25 users)	Impact at Beta (200+ users)
New HTTP connections refused	Players who happen to refresh get a brief error	Multiple players per deploy experience errors
Existing WebSocket connections dropped	All 25 testers disconnected simultaneously	All 200+ players disconnected — visible disruption
In-progress actions	Occasional lost chat	Trade transactions or

Failure Mode	Impact at Alpha (25 users)	Impact at Beta (200+ users)
interrupted	message	crafting timers corrupted

The solution is PM2 cluster mode with graceful shutdown signals, which enables **rolling restarts**: one instance is brought down gracefully while the other continues serving traffic, then the updated instance comes up before the second is restarted.

4.2 PM2 Cluster Mode for Rolling Restarts LINUX

SSH to the Oracle VM and update `/opt/havenworld/ecosystem.config.js`:

```
// /opt/havenworld/ecosystem.config.js // PM2 cluster mode – 2 instances for
zero-downtime rolling restarts module.exports = {  apps: [{    name:
'havenworld-server',    script: 'dist/index.js',    cwd:
'/opt/havenworld/apps/server',    // Cluster mode: PM2 manages 2 instances
via Node.js cluster module    instances: 2,    exec_mode: 'cluster',
autorestart: true,    watch: false,    // Per-instance memory limit: 1GB ×
2 = 2GB out of 24GB Oracle ARM RAM    max_memory_restart: '1G',    // Load
.env from the server directory    env_file:
'/opt/havenworld/apps/server/.env',    // Logging    error_file:
'/var/log/havenworld/error.log',    out_file:
'/var/log/havenworld/out.log',    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
merge_logs: true,    // Graceful shutdown configuration    kill_timeout:
5000,    // ms to wait for graceful shutdown before SIGKILL    wait_ready:
true,    // Wait for process.send('ready') before routing traffic
listen_timeout: 10000,    // ms to wait for the ready signal    // PM2
increments PORT for each instance: instance 0 = 3000, instance 1 = 3001
increment_var: 'PORT',    env: {    NODE_ENV: 'production',    PORT:
3000    }  }  ]};
```

Update `apps/server/src/index.ts` to emit the PM2 ready signal and implement graceful shutdown. Find the `httpServer.listen()` call and update it as follows:

```
// apps/server/src/index.ts  const PORT = Number(process.env.PORT) || 3000;
const start = async () => {  await prisma.$connect();
console.log('[Server] Database connected');  httpServer.listen(PORT,
'0.0.0.0', () => {    console.log(`[Server] HavenWorld instance listening on
port ${PORT}`);    // Signal PM2 that this instance is ready to receive
traffic.    // PM2 cluster mode waits for this signal before routing new
connections here.    // Without this, PM2 would route traffic before Express
is ready.    if (process.send) {    process.send('ready');
console.log(`[Server] Sent ready signal to PM2 (pid: ${process.pid})`);    }
});  });  // — Graceful Shutdown

const shutdown = async (signal:
string): Promise<void> => {  console.log(`[Server] ${signal} received –
starting graceful shutdown (pid: ${process.pid})`);    // 1. Stop accepting
new HTTP connections  httpServer.close(async () =>
{    console.log('[Server] HTTP server closed');    try {    // 2.
Close all Socket.io connections gracefully    io.close();
console.log('[Server] Socket.io closed');    // 3. Disconnect Prisma
```

```

connection pool      await prisma.$disconnect();      console.log('[Server]
Database disconnected');      // 4. Disconnect Redis client      await
redis.quit();      console.log('[Server] Redis disconnected');
console.log('[Server] Graceful shutdown complete');      process.exit(0);
} catch (err) {      console.error('[Server] Error during shutdown:', err);
process.exit(1);      });      // Safety net: force kill if graceful
shutdown takes longer than 5 seconds      setTimeout(() =>
{      console.error('[Server] Graceful shutdown timeout – forcing exit');
process.exit(1);      }, 5000); }); process.on('SIGTERM', () =>
shutdown('SIGTERM')); // PM2 sends SIGTERM during reload process.on('SIGINT',
() => shutdown('SIGINT')); // Ctrl+C in development start().catch((err) =>
{ console.error('[Server] Startup error:', err); process.exit(1); });

```

Apply the new PM2 configuration on the Oracle VM:

```

# [LINUX] SSH into Oracle VM first ssh ubuntu@$ORACLE_HOST # Navigate to the
project root cd /opt/havenworld # Pull the updated ecosystem.config.js and
server code git pull origin main cd apps/server && pnpm build && cd ../.. #
Delete the old single-instance PM2 entry and start with cluster mode pm2
delete havenworld-server 2>/dev/null || true pm2 start ecosystem.config.js #
Save the PM2 process list (survives VM reboots) pm2 save # Verify both
instances are running pm2 list

```

Expected output of pm2 list:

	id	name	namespace	mode	pid	status
cpu						
	0	havenworld-server	default	cluster	12345	online
0%	1	havenworld-server	default	cluster	12346	
online	0%					

Test the zero-downtime reload (this is what CI/CD will use instead of pm2 restart):

```

# [LINUX] – zero-downtime reload: restarts one instance at a time pm2 reload
ecosystem.config.js # Watch the log to observe the rolling restart sequence
pm2 logs havenworld-server --lines 20

```

4.3 Nginx Upstream for Load Balancing LINUX

With two PM2 instances running on ports 3000 and 3001, update Nginx to load balance across both. Edit the site configuration at

/etc/nginx/sites-available/havenworld:

```

# /etc/nginx/sites-available/havenworld # Upstream pool: both PM2 instances
upstream havenworld_backend {      least_conn;      # Route to
instance with fewest active connections      server 127.0.0.1:3000;      server
127.0.0.1:3001;      keepalive 32;      # Maintain 32 idle
connections to upstream (performance) } server {      listen 80;
server_name YOUR-IP-WITH-DASHES.nip.io;      # Redirect HTTP to HTTPS
return 301 https://$host$request_uri; } server {      listen 443 ssl;
server_name YOUR-IP-WITH-DASHES.nip.io;      ssl_certificate
/etc/letsencrypt/live/YOUR-DOMAIN/fullchain.pem;      ssl_certificate_key

```

```

/etc/letsencrypt/live/YOUR-DOMAIN/privkey.pem;    ssl_protocols TLSv1.2
TLSv1.3;    ssl_ciphers HIGH:!aNULL:!MD5;    # Rate limiting (configured
in nginx.conf http block)    limit_req zone=havenworld_limit burst=20
nodelay;    location / {    proxy_pass http://havenworld_backend;
proxy_http_version 1.1;    # WebSocket upgrade headers
proxy_set_header Upgrade $http_upgrade;    proxy_set_header Connection
"upgrade";    # Pass real client information to Express
proxy_set_header Host    $host;    proxy_set_header X-Real-IP
$remote_addr;    proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;    proxy_set_header X-Forwarded-Proto
$scheme;    # Extended timeouts for long-lived WebSocket connections
proxy_read_timeout 3600s;    proxy_send_timeout 3600s;
proxy_connect_timeout 10s;    } }

```

Test and reload Nginx:

```

# [LINUX] sudo nginx -t # Expected: nginx: configuration file
/etc/nginx/nginx.conf test is successful sudo systemctl reload nginx # No
restart needed – reload applies config changes with zero downtime

```

4.4 Socket.io Sticky Sessions MAC + LINUX

Socket.io's HTTP long-polling transport (the fallback before WebSocket upgrade completes) requires that all requests from a given client reach the same backend instance. This is called "sticky sessions." Since both PM2 instances are on the same VM, you can use the `@socket.io/cluster-adapter` instead of Redis Pub/Sub — it enables inter-process Socket.io broadcasting through Node.js IPC (inter-process communication), which is faster than Redis for same-machine clusters.

```

# [MAC] Install the cluster adapter pnpm add @socket.io/cluster-adapter --
filter server

```

Update `apps/server/src/index.ts` to set up the cluster adapter in production:

```

// apps/server/src/index.ts import { createServer } from 'http'; import
{ Server } from 'socket.io'; import { setupWorker } from '@socket.io/cluster-
adapter'; const httpServer = createServer(app); const io = new
Server(httpServer, { cors: { origin: process.env.CLIENT_URL, credentials:
true }, // Enable Socket.io sticky session cookie (required for polling
fallback) cookie: true, }); // Enable cluster adapter in production (same-
VM multi-instance broadcasting) if (process.env.NODE_ENV === 'production') {
setupWorker(io); console.log('[Server] Socket.io cluster adapter enabled
(IPC broadcasting)'); }

```

Additionally, enable Nginx IP hash to ensure a client's HTTP long-polling requests always reach the same upstream instance. Update the upstream block to add `ip_hash`:

```

upstream havenworld_backend {    ip_hash;    # Sticky: client IP
always routes to same instance    server 127.0.0.1:3000;    server
127.0.0.1:3001;    keepalive 32; }

```

💡 `ip_hash` vs. `least_conn`

Use `ip_hash` when Socket.io is in play (required for polling fallback). Use `least_conn` for pure WebSocket connections (WebSocket is a single long-lived TCP connection that doesn't need stickiness). Since both transports are possible, `ip_hash` is the safer choice for HavenWorld.

4.5 Updated Deployment Script MAC + LINUX

Update `.github/workflows/deploy-server.yml` to use `pm2 reload` instead of `pm2 restart`:

```
- name: SSH Deploy to Oracle VM (Zero-Downtime)      uses: appleboy/ssh-
action@v1.0.3      with:      host: ${ secrets.ORACLE_HOST }}
username: ubuntu      key: ${ secrets.ORACLE_SSH_KEY }}
timeout: 300s      script: |      set -e      cd
/opt/havenworld      echo "📦 Pulling latest code..."      git
pull origin main      echo "📦 Installing dependencies..."
pnpm install --frozen-lockfile      echo "🗄️ Running database
migrations..."      cd apps/server      npx prisma migrate
deploy      echo "🔨 Building server..."      pnpm build
cd /opt/havenworld      echo "🔄 Rolling reload (zero-downtime)..."
# pm2 reload restarts instances one at a time      # --update-env
picks up any .env changes      pm2 reload ecosystem.config.js --
update-env      # Save process list in case of VM reboot
pm2 save      # Give instances time to settle      sleep 4
# Verify deployment: at least one instance must be online
ONLINE=$(pm2 jlist | python3 -c "      import json,sys
procs = json.load(sys.stdin)      online = sum(1 for p in procs if
p.get('pm2_env',{}).get('status')=='online')      print(online)
")      echo "Instances online: $ONLINE"      if [ "$ONLINE" -
lt 1 ]; then      echo "❌ Deployment failed – no instances online"
exit 1      fi      echo "✅ Deployment successful – $ONLINE
instance(s) online"
```

4.6 Health Check Endpoint Enhancement MAC

Update the `/health` endpoint in `apps/server/src/index.ts` to include cluster information. The `pid` field is useful for debugging load balancing — you can call `/health` multiple times and observe different PIDs as Nginx distributes requests across instances:

```
// apps/server/src/index.ts app.get('/health', async (_req, res) => { try {
// Verify database is reachable      await prisma.$queryRaw`SELECT 1`;      //
Verify Redis is reachable      const redisPong = await redis.ping();      const
redisAlive = redisPong === 'PONG';      res.status(200).json({      status:
'ok',      timestamp: new Date().toISOString(),      uptime:
Math.floor(process.uptime()),      // seconds since this instance started
pid:      process.pid,      // PM2 instance PID (useful
for debugging)      instance: process.env.pm_id ?? 'standalone', // PM2
instance ID (0 or 1)      memory: {      heapUsed:
Math.round(process.memoryUsage().heapUsed / 1024 / 1024) + 'MB',
heapTotal: Math.round(process.memoryUsage().heapTotal / 1024 / 1024) + 'MB',
rss:      Math.round(process.memoryUsage().rss / 1024 / 1024) + 'MB',
},      database: 'connected',      redis:      redisAlive ? 'connected' :
'error',      version: process.env.npm_package_version ??
'unknown',      }); } catch (err) {      res.status(503).json({      status:
'error',      timestamp: new Date().toISOString(),      pid:
process.pid,      message: String(err),      }); } });
```

Test by calling the endpoint twice and observing different pid values (confirming Nginx is distributing requests):

```
# [MAC] – run twice, observe that pid alternates between the two PM2
instances curl -s https://[oracle-ip].nip.io/health | jq '{status, pid,
instance, uptime}' curl -s https://[oracle-ip].nip.io/health | jq '{status,
pid, instance, uptime}'
```

4.7 Deployment Runbook MAC + LINUX

Create docs/DEPLOY.md at the repository root. This is the authoritative deployment reference for all contributors:

```
# HavenWorld Deployment Runbook Last updated: 2026-09-18 by Daniel --- ##
Standard Deploy – Push to Main (Automated) 1. On Mac: develop your feature
on a branch 2. Push branch: `git push origin feature/my-feature` 3. Open PR:
your branch → main 4. Wait for `pr-check.yml` to pass (lint, type check,
tests, build) 5. Merge the PR 6. GitHub Actions automatically:      - Deploys
client → Cloudflare Pages (~2 minutes)      - Syncs any new 3D assets →
Cloudflare R2 (if .glb files changed)      - Deploys server → Oracle VM via
SSH, rolling reload (~3 minutes) ## Verifying a Deployment After GitHub
Actions completes, verify: - [ ] https://havenworld.pages.dev loads
(Cloudflare Pages) - [ ] https://[ip].nip.io/health returns {"status":"ok"}
(Oracle VM) - [ ] pm2 list shows 2 instances with status "online" - [ ] No
error spike in pm2 logs havenworld-server --lines 50 ## Emergency Rollback
(Server) ssh ubuntu@[ORACLE_IP] cd /opt/havenworld git log --oneline -10
# Find the last known-good commit git checkout [COMMIT_HASH]      # Check out
that commit cd apps/server pnpm build      # Rebuild server cd
/opt/havenworld pm2 reload ecosystem.config.js # Rolling reload with old code
# Monitor logs: pm2 logs havenworld-server --lines 30 ## Database Migration
Rollback IMPORTANT: Never auto-rollback database migrations. Assess the
migration manually in psql, understand the exact change, and reverse it
manually. Document every step in the #incident-log Discord channel. ##
Deployment Environment Checklist Before any major deploy (e.g., beta
launch): - [ ] ALPHA_INVITE_ONLY=false in /opt/havenworld/apps/server/.env -
[ ] VITE_ASSET_BASE_URL set in Cloudflare Pages environment variables - [ ]
```

R2 bucket havenworld-assets is publicly accessible - [] pm2 save run after any ecosystem.config.js change - [] SSL certificate has > 60 days remaining

Section 5 — Part 9A Completion Checklist

Mark each item as complete before proceeding to Part 9B. All items are required. Record the date each item is completed in `LAUNCH_GATE.md`.

Alpha-to-Beta Gate

- ☐ `scripts/beta-gate-check.sh` created, executable, and passing with zero failures
- ☐ `LAUNCH_GATE.md` created at repo root; all 4 gate categories fully documented
- ☐ k6 load test passing at 50 concurrent virtual users (p95 < 500ms, zero errors)
- ☐ Gate Category 1 (Infrastructure Stability) — all 10 items verified and signed off
- ☐ Gate Category 2 (Application Quality) — all 9 items verified and signed off
- ☐ Gate Category 3 (Game Loop Quality) — all 9 items verified by 3+ named testers
- ☐ Gate Category 4 (Community & Legal) — all 9 items verified and signed off
- ☐ `ALPHA_INVITE_ONLY=false` set in production `.env` on Oracle VM
- ☐ Registration form updated — invite code input field removed from `index.html`
- ☐ `auth.ts` `register()` updated — `inviteCode` parameter removed
- ☐ Launch announced in Discord `#announcements` and on Hashnode

Progressive Web App

- ☐ `apps/client/public/manifest.webmanifest` created with all required fields
- ☐ Manifest linked in `index.html` with all PWA meta tags (including Apple-specific tags)

- ☐ `apps/client/public/icons/icon-512.png` created (512×512, maskable safe zone)
- ☐ `apps/client/public/icons/icon-192.png` created via `sips` resize
- ☐ Icon previewed at `maskable.app` — logo fully visible in all clip shapes
- ☐ `vite-plugin-pwa` installed and added to `vite.config.ts`
- ☐ Workbox runtime caching configured for `.glb` (30-day CacheFirst) and `.wasm` (90-day CacheFirst)
- ☐ `apps/client/public/offline.html` created and referenced in Workbox config
- ☐ `InstallPrompt.ts` created at `apps/client/src/ui/InstallPrompt.ts`
- ☐ PWA install banner HTML added to `index.html`
- ☐ `InstallPrompt` initialized in `main.ts` after login
- ☐ `apps/client/public/_headers` created with MIME types, cache rules, and security headers
- ☐ Chrome DevTools → Application → Manifest shows no errors
- ☐ Service worker registered and status shows "activated and running"
- ☐ Lighthouse PWA audit score ≥ 90 (target: 100)
- ☐ PWA installs successfully as standalone app on Chrome desktop

Cloudflare R2 CDN

- ☐ R2 bucket `havenworld-assets` created in Cloudflare Dashboard
- ☐ Public access enabled on the bucket
- ☐ Custom domain `assets.havenworld.pages.dev` configured and resolving
- ☐ R2 API token created with Object Read and Write scope on `havenworld-assets`
- ☐ GitHub Secrets added: `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`
- ☐ `rclone` installed on Mac via Homebrew
- ☐ `rclone` configured with remote name `havenworld-r2`
- ☐ `rclone ls havenworld-r2:havenworld-assets` returns successfully (exit code 0)

- ☐ `scripts/upload-assets.sh` created and tested — assets visible in R2 Dashboard
- ☐ `scripts/download-assets.sh` created and documented in README.md
- ☐ `upload:assets` and `download:assets` scripts added to root `package.json`
- ☐ `.gitignore` updated to exclude all GLB and tileset PNG files
- ☐ GLB files removed from Git history via `git filter-repo` (if previously committed)
- ☐ `apps/client/src/world/RoomLoader.ts` updated to use `VITE_ASSET_BASE_URL`
- ☐ `.env.development` and `.env.production` updated with `VITE_ASSET_BASE_URL`
- ☐ Cloudflare Pages environment variable `VITE_ASSET_BASE_URL` set for Production
- ☐ R2 CI/CD step added to `deploy-client.yml`
- ☐ Room load time verified in browser Network tab: < 3 seconds from R2 CDN

Zero-Downtime Deployment

- ☐ `ecosystem.config.js` updated: `instances: 2`, `exec_mode: 'cluster'`, `increment_var: 'PORT'`
- ☐ `process.send('ready')` added to `httpServer.listen()` callback in `index.ts`
- ☐ SIGTERM and SIGINT graceful shutdown handlers implemented in `index.ts`
- ☐ Graceful shutdown tested: `kill -SIGTERM [pid]` — server closes cleanly (exit 0)
- ☐ Nginx `upstream havenworld_backend` block configured for ports 3000 and 3001
- ☐ Nginx config tested with `sudo nginx -t` and reloaded with `sudo systemctl reload nginx`
- ☐ `@socket.io/cluster-adapter` installed and `setupWorker(io)` called in production
- ☐ `pm2 reload ecosystem.config.js` tested — confirmed no dropped WebSocket connections
- ☐ `deploy-server.yml` updated to use `pm2 reload` instead of `pm2 restart`
- ☐ `/health` endpoint returns `pid`, `instance`, and `memory` fields
- ☐ Two separate PIDs observed in consecutive `/health` calls (load balancing confirmed)

- ☐ `pm2 save` run after cluster mode activated (survives Oracle VM reboots)
- ☐ `docs/DEPLOY.md` created and reviewed by Daniel

What's in Part 9B

Coming Next: Part 9B — Completing Launch Readiness

Part 9B finishes the Launch Readiness phase with five remaining systems:

- **Self-hosted analytics dashboard** (Umami or Plausible on free tier) — see how many players join, where they drop off, which rooms are most popular, all without Google Analytics
- **Player onboarding tutorial room** — a scripted first-time user experience (FTUE) that walks new players through avatar customization, chat, trading, and room navigation in a guided environment
- **Live monitoring upgrades** — Prometheus metrics endpoint on the Express server + Grafana dashboard on Oracle VM (both free tier) for real-time CPU, memory, WebSocket connection count, and Socket.io event rate graphs
- **Seasonal event activation system** — a database-driven feature flag system allowing events (Halloween decorations, holiday loft themes, bonus coin weekends) to be toggled on/off without code deploys
- **"What's New" patch notes modal** — a version-aware modal that appears once after each deploy to inform returning players of new features and bug fixes

After Part 9B, only Part 10 remains: Post-Launch Operations (incident response, community growth, database backups, and the Part 10 series retrospective).

HavenWorld Build Guide — Part 9A of 10 • GPL-3.0 Open Source • Free-tier only •
Authored by Daniel • Anoka, MN • 18 September 2026 • Stack: Babylon.js 7 • Vite 6
• TypeScript • Express 5 • Socket.io 4 • Prisma • PostgreSQL 16 • Redis 7 •
Cloudflare Pages • Cloudflare R2 • Oracle Cloud ARM • PM2 • Nginx